

DATA-260 - Homework 5

Rachandeep Kaur

GitHub: <https://github.com/rachandeepkaur/data260-7117>

Section 0 — Personal Configuration and Domain

Value	Definition	My value
SID4	Last four digits of SJSU Student ID	7117
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	$8000 + 817 = 8817$
PREFIX	"s" + SID4	s7117
SEED	SID4	7117
VERIFY_SEED	$260000 + \text{SID4}$	267117
DOMAIN_ID	$\text{SID4} \bmod 8$	5 (Local restaurant inspections)

Item	Value
Hardware	MacBook Pro — Intel Core i9-9880H @ 2.30 GHz (8 cores / 16 threads), 32 GB RAM, macOS 26.6.2
Local model	qwen3:8b via Ollama (temperature 0, seed 7117, thinking disabled)
Embedding model	BAAI/bge-small-en-v1.5 (fastembed / ONNXRuntime, 384-dim, 512-token window)
Database	MySQL 8.4.11 (Docker image mysql:8.4, container s7117-mysql), database s7117_rel
Backend/frontend	FastAPI + SQLAlchemy 2 on port 8817; React 18 + React Router 6 (Vite) on port 5173
Tagged commit	hw5 Commit -

Part 1 - FastAPI + MySQL and Redux Client

Primary entity: inspections (unique inspection_code, score default 100, inspector_id FK).
Related entity: inspectors (full_name, district, unique email). Deleting an inspector who still has inspections is blocked (ON DELETE RESTRICT → 409).

I. Database

```
***** 1. row *****
Table: inspectors
Create Table: CREATE TABLE `inspectors` (
  `id` int NOT NULL AUTO_INCREMENT,
  `full_name` varchar(120) NOT NULL,
  `district` varchar(120) NOT NULL,
  `email` varchar(255) NOT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_inspectors_email` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=21 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
id      full_name      district      email      created_at      updated_at
1      Linh Nguyen      Alum Rock      linh.nguyen@sccgov-eh.org      2026-10-05 04:44:48      2026-10-05 04:44:48
2      Daniel Ramos      Almaden Valley      daniel.ramos@sccgov-eh.org      2026-10-05 04:44:48      2026-10-05 04:44:48
3      Linh Shah      Cambrian      linh.shah@sccgov-eh.org      2026-10-05 04:44:48      2026-10-05 04:44:48
4      Maria Ramos      North San Jose      maria.ramos@sccgov-eh.org      2026-10-05 04:44:48      2026-10-05 04:44:48
5      Mei Lopez      Alum Rock      mei.lopez@sccgov-eh.org      2026-10-05 04:44:48      2026-10-05 04:44:48
```

MySQL database s7117_rel

II. API — Postman

 **http://127.0.0.1:8817/api/inspectors**

POST

▼

http://127.0.0.1:8817/api/inspectors

ParamsAuthorizationHeaders (9)Body ●Pre-request ScriptTestsSettings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

JSON ▼

```
1 {
2   "full_name": "Rachandeep Kaur",
3   "district": "Downtown San Jose",
4   "email": "rachandeep.kaur@sccgov-eh.org"
5 }
6
```

BodyCookies (1)Headers (4)Test Results

PrettyRawPreviewVisualizeJSON ▼

```
1 {}
2   "id": 25,
3   "full_name": "Rachandeep Kaur",
4   "district": "Downtown San Jose",
5   "email": "rachandeep.kaur@sccgov-eh.org",
6   "created_at": "2026-10-05T18:27:49",
7   "updated_at": "2026-10-05T18:27:49"
8 }
```

Create inspector — POST /api/inspectors

HTTP http://127.0.0.1:8817/api/inspectors?page=1&page_size=5 Save </>

GET http://127.0.0.1:8817/api/inspectors?page=1&page_size=5 Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
☒	page	1	
☒	page_size	5	
	Key	Value	

Body Cookies (1) Headers (5) Test Results 200 OK 33 ms 969 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "full_name": "Linh Nguyen",
4   "district": "Alum Rock",
5   "email": "linh.nguyen@sccgov-eh.org",
6   "created_at": "2026-10-05T04:44:48",
7   "updated_at": "2026-10-05T04:44:48"
8 },
9 {
10  "id": 2,
```

List inspectors (pagination) — GET /api/inspectors?page=1&page_size=5

HTTP <http://127.0.0.1:8817/api/inspectors/3> S

GET <http://127.0.0.1:8817/api/inspectors/3> Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk E
	Key	Value	

Body Cookies (1) Headers (4) Test Results 200 OK 16 ms 284 B Save Respon

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "full_name": "Linh Shah",
4   "district": "Cambrian",
5   "email": "linh.shah@sccgov-eh.org",
6   "created_at": "2026-10-05T04:44:48",
7   "updated_at": "2026-10-05T04:44:48"
8 }
```

Get inspector — GET /api/inspectors/{id} → 200

HTTP <http://127.0.0.1:8817/api/inspectors/25> Save

PUT <http://127.0.0.1:8817/api/inspectors/25> Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

● none ● form-data ● x-www-form-urlencoded ● raw ● binary JSON

```
1 {
2   "full_name": "Rachandeep Kaur",
3   "district": "Redwood City",
4   "email": "rachandeep.kaur@sccgov.org"
5 }
6
```

Body Cookies (1) Headers (4) Test Results 200 OK 61 ms 298 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 25,
3   "full_name": "Rachandeep Kaur",
4   "district": "Redwood City",
5   "email": "rachandeep.kaur@sccgov.org",
6   "created_at": "2026-10-05T18:27:49",
7   "updated_at": "2026-10-05T18:33:09"
8 }
```

Update inspector — PUT /api/inspectors/{id} → 200

HTTP <http://127.0.0.1:8817/api/records> Save

POST <http://127.0.0.1:8817/api/records> Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

● none ● form-data ● x-www-form-urlencoded ● raw ● binary JSON

```
1 {
2   "inspection_code": "INS-005006",
3   "facility_name": "FA0206933 - YUMMY KITCHEN",
4   "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
5   "score": 94,
6   "inspector_id": 25
7 }
8
```

Body Cookies (1) Headers (4) Test Results 201 Created 110 ms 372 B Save R

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 5006,
3   "inspection_code": "INS-005006",
4   "facility_name": "FA0206933 - YUMMY KITCHEN",
5   "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
6   "score": 94,
7   "inspector_id": 25,
8   "created_at": "2026-10-05T18:50:54",
9   "updated_at": "2026-10-05T18:50:54"
10 }
```

Create inspection — POST /api/records → 201

HTTP <http://127.0.0.1:8817/api/records> Save

GET <http://127.0.0.1:8817/api/records> Send

Params Authorization Headers (9) Body ● Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
	Key	Value	

Body Cookies (1) Headers (5) Test Results 200 OK 294 ms 1.14 MB Save Response

Pretty Raw Preview Visualize JSON ⌵

```
1 {
2   {
3     "id": 5006,
4     "inspection_code": "INS-005006",
5     "facility_name": "FA0206933 - YUMMY KITCHEN",
6     "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
7     "score": 94,
8     "inspector_id": 25,
9     "created_at": "2026-10-05T18:50:54",
10    "updated_at": "2026-10-05T18:50:54"
11  },
12  {
13    "id": 5000,
14    "inspection_code": "INS-005000",
15    "facility_name": "FA0206933 - YUMMY KITCHEN",
16    "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
17    "score": 94,
18    "inspector_id": 25,
19    "created_at": "2026-10-05T18:50:54",
20    "updated_at": "2026-10-05T18:50:54"
```

List inspections — GET /api/records → 200

HTTP <http://127.0.0.1:8817/api/records/5006>

GET <http://127.0.0.1:8817/api/records/5006>

Params Authorization Headers (9) Body ● Pre-request Script Tests Settings

Query Params

	Key	Value
	Key	Value

Body Cookies (1) Headers (4) Test Results 200 OK 13 ms 367

Pretty Raw Preview Visualize JSON ⌵

```
1 {
2   "id": 5006,
3   "inspection_code": "INS-005006",
4   "facility_name": "FA0206933 - YUMMY KITCHEN",
5   "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
6   "score": 94,
7   "inspector_id": 25,
8   "created_at": "2026-10-05T18:50:54",
9   "updated_at": "2026-10-05T18:50:54"
```

Get inspection — GET /api/records/{id} → 200

HTTP <http://127.0.0.1:8817/api/records/5006>

PUT <http://127.0.0.1:8817/api/records/5006>

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON

```
1  {
2    "id": 5006,
3    "inspection_code": "INS-005006",
4    "facility_name": "FA0206933 - YUMMY KITCHEN",
5    "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
6    "score": 94,
7    "inspector_id": 20,
8    "created_at": "2026-10-05T18:50:54",
9    "updated_at": "2026-10-05T18:50:54"
10 }
```

Body Cookies (1) Headers (4) Test Results 200 OK 25 ms 367 B **Sa**

Pretty Raw Preview Visualize JSON

```
1  {
2    "id": 5006,
3    "inspection_code": "INS-005006",
4    "facility_name": "FA0206933 - YUMMY KITCHEN",
5    "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
6    "score": 94,
7    "inspector_id": 20,
8    "created_at": "2026-10-05T18:50:54",
9    "updated_at": "2026-10-05T18:55:23"
10 }
```

Update inspection — PUT /api/records/{id} → 200

HTTP <http://127.0.0.1:8817/api/inspectors/25/inspections>

GET <http://127.0.0.1:8817/api/inspectors/25/inspections> Send

Params Authorization Headers (9) Body ● Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies (1) Headers (5) Test Results 200 OK 15 ms 387 B Save Response

Pretty Raw Preview Visualize JSON ↕

```
1 {
2   {
3     "id": 5006,
4     "inspection_code": "INS-005006",
5     "facility_name": "FA0206933 - YUMMY KITCHEN",
6     "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
7     "score": 94,
8     "inspector_id": 25,
9     "created_at": "2026-10-05T18:50:54",
10    "updated_at": "2026-10-05T18:57:21"
11  }
12 }
```

Relationship query — GET /api/inspectors/{id}/inspections → 200

HTTP <http://127.0.0.1:8817/api/inspectors/25>

DELETE <http://127.0.0.1:8817/api/inspectors/25> Send

Params Authorization Headers (9) Body ● Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies (1) Headers (4) Test Results 409 Conflict 24 ms 213 B Save Response

Pretty Raw Preview Visualize JSON ↕

```
1 {
2   "detail": "Inspector 25 still has 1 inspection(s); reassign or delete them first"
3 }
```

Delete inspector that still has inspections → 409

HTTP **http://127.0.0.1:8817/api/records**

POST **http://127.0.0.1:8817/api/records**

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▾

```
1 {
2   "inspection_code": "12345",
3   "facility_name": "TEST CAFE",
4   "site_address": "1 TEST ST, SAN JOSE, CA 95112",
5   "inspector_id": 1
6 }
7
```

Body Cookies (1) Headers (4) Test Results  422 Unprocessable Content 14 ms 3:

Pretty Raw Preview Visualize **JSON** ▾ 

```
1 {
2   "detail": [
3     {
4       "type": "string_pattern_mismatch",
5       "loc": [
6         "body",
7         "inspection_code"
8       ],
9       "msg": "String should match pattern '^INS-\\d{6}$'",
10      "input": "12345",
11      "ctx": {
12        "pattern": "^INS-\\d{6}$"
13      }
14     }
15   ]
16 }
```

Validation error: invalid email / bad inspection_code format → 422

HTTP **http://127.0.0.1:8817/api/records**

POST **http://127.0.0.1:8817/api/records**

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

● none ● form-data ● x-www-form-urlencoded ● **raw** ● binary **JSON** ▾

```
1 {
2   "inspection_code": "INS-000001",
3   "facility_name": "TEST CAFE",
4   "site_address": "1 TEST ST, SAN JOSE, CA 95112",
5   "inspector_id": 1
6 }
7
```

Body Cookies (1) Headers (4) Test Results 409 Conflict 55 ms 185 B

Pretty Raw Preview Visualize **JSON** ▾

```
1 {
2   "detail": "inspection_code INS-000001 already exists"
3 }
```

Constraint violation: duplicate inspection_code → 409

HTTP **http://127.0.0.1:8817/api/records/9999**

GET **http://127.0.0.1:8817/api/records/9999**

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

● none ● form-data ● x-www-form-urlencoded ● **raw** ● binary **JSON** ▾

```
1 {
2   "inspection_code": "12345",
3   "facility_name": "TEST CAFE",
4   "site_address": "1 TEST ST, SAN JOSE, CA 95112",
5   "inspector_id": 1
6 }
7
```

Body Cookies (1) Headers (4) Test Results 404 Not Found 14 ms 174 B

Pretty Raw Preview Visualize **JSON** ▾

```
1 {
2   "detail": "Record with id 9999 not found"
3 }
```

Not found — GET /api/records/999999 → 404

HTTP <http://127.0.0.1:8817/api/records/5006>

DELETE <http://127.0.0.1:8817/api/records/5006>

Params Authorization Headers (9) Body ● Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies (1) Headers (4) Test Results 200 OK 25 ms

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 5006,
3   "inspection_code": "INS-005006",
4   "facility_name": "FA0206933 - YUMMY KITCHEN",
5   "site_address": "1711 BRANHAM LN A9, SAN JOSE, CA 95118",
6   "score": 94,
7   "inspector_id": 25,
8   "created_at": "2026-10-05T18:50:54",
9   "updated_at": "2026-10-05T18:57:21"
10 }
```

Delete inspection — DELETE /api/records/{id} → 200

III. Redux Client

```
export const fetchInspections = createAsyncThunk(
  "inspections/fetch",
  async ({ page = 0, pageSize = 25 } = {}, { rejectWithValue }) => {
    try {
      const res = await http.get("/api/records", { params: { limit: pageSize, offset: page * pageSize } });
      return { items: res.data, total: Number(res.headers["x-total-count"] ?? res.data.length), page };
    } catch (err) {
      return rejectWithValue(toError(err));
    }
  }
);
```

```
export default function Home() {
  const dispatch = useDispatch();
  const { items, total, page, pageSize, status, error } = useSelector((state) => state.inspections);
  const inspectors = useSelector((state) => state.inspectors.items);
```

restaurant inspections

Home

Add Inspection Record

Update by ID

Signed in as Rachandeep Kaur

Logout

Inspection Records

5,000 records in MySQL

ID	Code	Facility Name / ID	Program Site Address	Score	Inspector	Actions
5000	INS-005000	FA0470962 - LA CANTINA	8859 WINCHESTER BLVD, SAN JOSE, CA 95127	100	Linh Nguyen	EditDelete
4999	INS-004999	FA0422276 - RED BURGERS	4407 ALMADEN EXPY, SAN JOSE, CA 95128	100	Maria Ramos	EditDelete
4998	INS-004998	FA0536430 - LA PIZZERIA	3057 MCKEE RD, SAN JOSE, CA 95120	100	Tara Shah	EditDelete
4997	INS-004997	FA0599994 - LA CURRY HOUSE	6119 MONTEREY RD, SAN JOSE, CA 95123	100	Tara Chen	EditDelete
4996	INS-004996	FA0179358 - VALLEY WOK	8078 SENTER RD, SAN JOSE, CA 95117	100	Aisha Shah	EditDelete
4995	INS-004995	FA0907630 - MISSION DINER	6780 WINCHESTER BLVD, SAN JOSE, CA 95116	100	Kevin Wong	EditDelete
4994	INS-004994	FA0272023 - BAY RAMEN	951 E SANTA CLARA ST, SAN JOSE, CA 95136	100	Ravi Lee	EditDelete
4993	INS-004993	FA0162094 - GREEN RAMEN	2378 SARATOGA AVE, SAN JOSE, CA 95128	100	Daniel Ramos	EditDelete

Home — store reads records

```
export const createInspection = createAsyncThunk("inspections/create", async (record, { rejectWithValue }) => {
  try {
    return (await http.post("/api/records", record)).data;
  } catch (err) {
    return rejectWithValue(toError(err));
  }
});
```

```
.addCase(createInspection.fulfilled, (state, { payload }) => {
  state.items.unshift(payload);
  if (state.items.length > state.pageSize) state.items.pop();
  state.total += 1;
  state.error = null;
})
```

Add Inspection Record

Inspection Code

INS-005008

Facility Name / ID

FA0717117 - REDUX DEMO KITCHEN

Program Site Address

1 WASHINGTON SQ, SAN JOSE, CA 95192

Score (0-100)

91

Inspector

25 - Rachandeep Kaur (Redwood City)



Add Inspection Record

Inspection Records

5,001 records in MySQL

ID	Code	Facility Name / ID	Program Site Address	Score	Inspector	Actions
5008	INS-005008	FA0717117 - REDUX DEMO KITCHEN	1 WASHINGTON SQ, SAN JOSE, CA 95192	91	Rachandeep Kaur	EditDelete

Create — new record added to Home

```
export const updateInspection = createAsyncThunk(
  "inspections/update",
  async ({ id, changes }, { rejectWithValue }) => {
    try {
      return (await http.put(`/api/records/${id}`, changes)).data;
    } catch (err) {
      return rejectWithValue(toError(err));
    }
  }
);
```

```
.addCase(updateInspection.fulfilled, (state, { payload }) => {
  const i = state.items.findIndex((r) => r.id === payload.id);
  if (i !== -1) state.items[i] = payload;
  state.error = null;
})
```

Update Inspection Record #5008

Inspection Code

Facility Name / ID

Program Site Address

Score (0-100)

Inspector



Update Inspection Record

Inspection Records

5,001 records in MySQL

ID	Code	Facility Name / ID	Program Site Address	Score	Inspector	Actions
5008	INS-005008	FA0717117 - REDUX DEMO KITCHEN	1 WASHINGTON SQ, SAN JOSE, CA 95192	67	Rachandeep Kaur	<button>Edit</button> <button>Delete</button>

Update — record changed by ID

```
export const deleteInspection = createAsyncThunk("inspections/delete", async (id, { rejectWithValue }) => {
  try {
    await http.delete(`/api/records/${id}`);
    return id;
  } catch (err) {
    return rejectWithValue(toError(err));
  }
});
```

```
.addCase(deleteInspection.fulfilled, (state, { payload: id }) => {
  state.items = state.items.filter((r) => r.id !== id);
  state.total -= 1;
  state.error = null;
})
```

Inspection Records

5,000 records in MySQL

ID	Code	Facility Name / ID	Program Site Address	Score	Inspector	Actions
5000	INS-005000	FA0470962 - LA CANTINA	8859 WINCHESTER BLVD, SAN JOSE, CA 95127	100	Linh Nguyen	<button>Edit</button> <button>Delete</button>

Delete without page reload — row 5008 removed from Home

Part 2 - MCP Tool Servers

A. TheMealDB server (meals)

search_meals_by_name

Search meals by name. Returns up to 'limit' meals: id, name, area, category, thumb.

✓ Read-only

✗ Destructive

✓ Idempotent

✓ Open-world

query *

Arrabiata

limit

5

Tool-specific Metadata:

No metadata pairs.

Add Pair

Output Schema:

type: "object"
additionalProperties: true
title: "search_meals_by_nameDict0Output"
}

Expand

Run Tool

Copy Input

Tool Result: Success

Structured Content:

```
{  
  count: 1  
  results: [  
    0: {  
      id: "52771"  
      name: "Spicy Arrabiata Penne"  
      area: "Italian"  
      category: "Vegetarian"  
      thumb: "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"  
    }  
  ]  
  message: null  
}
```

✓ Valid according to output schema

Unstructured Content:

✓ Structured content matches text block

```
{  
  count: 1  
  results: [  
    0: {  
      id: "52771"  
      name: "Spicy Arrabiata Penne"  
      area: "Italian"  
      category: "Vegetarian"  
      thumb: "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg"  
    }  
  ]  
  message: null  
}
```

search_meals_by_name (query = Arrabiata, limit = 5)

meals_by_ingredient

Filter meals by main ingredient. Returns small cards: id, name, thumb.

✓ Read-only

✗ Destructive

✓ Idempotent

✓ Open-world

ingredient +

chicken

limit

12

Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

```
type: "object"
additionalProperties: true
title: "meals_by_ingredientDictOutput"
}
```

Run Tool

Copy Input

Tool Result: Success

Structured Content:

```
{
  count: 12
  results: [
    0: {
      id: "52940"
      name: "Brown Stew Chicken"
      thumb: "https://www.themeadb.com/images/media/meals/syxpqx1515365095.jpg"
    }
    1: {
      id: "53161"
      name: "Chicken & chorizo rice pot"
      thumb: "https://www.themeadb.com/images/media/meals/fk88jp1763288767.jpg"
    }
    2: {
      id: "52846"
      name: "Chicken & mushroom Hotpot"
      thumb: "https://www.themeadb.com/images/media/meals/uuuuqp1511297845.jpg"
    }
    3: {
      id: "52796"
      name: "Chicken Alfredo Primavera"
      thumb: "https://www.themeadb.com/images/media/meals/syqypv1486981727.jpg"
    }
    4: {
      id: "52934"
      name: "Chicken Basquaise"
      thumb: "https://www.themeadb.com/images/media/meals/wrvuqv1511888994.jpg"
    }
    5: {
      id: "52956"
      name: "Chicken Congee"
      thumb: "https://www.themeadb.com/images/media/meals/1529448352.jpg"
    }
    6: {
      id: "52795"
      name: "Chicken Nandi"
      thumb: "https://www.themeadb.com/images/media/meals/vyxwsp1486979827.jpg"
    }
  ]
}
```

meals_by_ingredient (ingredient = chicken)

id *

Copy JSON

Format JSON

52771

Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

```
type: "object"
additionalProperties: true
title: "meal_detailsDictOutput"
}
```

Run Tool

Copy Input

Tool Result: Success

Structured Content:

```
{
  id: "52771"
  name: "Spicy Arrabiata Penne"
  category: "Vegetarian"
  area: "Italian"
  instructions: "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then add the pasta. Cook according to the package instructions, about 9 minutes. In a large skillet over medium-high heat, add the olive oil and heat until the oil starts to shimmer. Add the garlic and cook, stirring, until fragrant, 1 to 2 minutes. Add the chopped tomatoes, red chile flakes, Italian seasoning and salt and pepper to taste. Bring to a boil and cook for 5 minutes. Remove from the heat and add the chopped basil. Drain the pasta and add it to the sauce. Garnish with Parmigiano-Reggiano flakes and more basil and serve warm."
  image: "https://www.themealdb.com/images/media/meals/ustsqw1468258014.jpg"
  source: null
  youtube: "https://www.youtube.com/watch?v=IIszT_guI08"
  ingredients: [
    0: {
      name: "penne rigate"
      measure: "1 pound"
    }
    1: {
      name: "olive oil"
      measure: "1/4 cup"
    }
    2: {
      name: "garlic"
      measure: "3 cloves"
    }
    3: {
      name: "chopped tomatoes"
      measure: "1 tin"
    }
    4: {
      name: "red chilli flakes"
      measure: "1/2 teaspoon"
    }
    5: {
      name: "italian seasoning"
      measure: "1/2 teaspoon"
    }
  ]
}
```

meal_details (id = 52771)

random_meal

One random meal, same shape as meal_details.

Read-only

Destructive

Idempotent

Open-world

Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

type: "object"

additionalProperties: true

title: "random_mealDictOutput"

}

Run Tool

Copy Input

Tool Result: Success

Structured Content:

{

id: "52826"

name: "Braised Beef Chilli"

category: "Beef"

area: "Mexican"

instructions: "Preheat the oven to 120C/225F/gas mark 1.

Take the meat out of the fridge to de-chill. Pulse the onions and garlic in a food processor until finely chopped. Heat 2 tbsp olive oil in a large casserole and sear the meat on all sides until golden.

Set to one side and add another small slug of oil to brown the chorizo. Remove and add the onion and garlic, spices, herbs and chillies then cook until soft in the chorizo oil. Season with salt and pepper and add the vinegar, tomatoes, ketchup and sugar.

Put all the meat back into the pot with 400ml water (or red wine if you prefer), bring up to a simmer and cook, covered, in the low oven.

After 2 hours, check the meat and add the beans. Cook for a further hour and just before serving, pull the meat apart with a pair of forks."

image: "https://www.themealdb.com/images/media/meals/uuqvwu1504629254.jpg"

source: "http://www.telegraph.co.uk/food-and-drink/recipes/braised-beef-chilli-con-carne/"

youtube: "https://www.youtube.com/watch?v=z4k5oJgsu6Y"

ingredients: [

0: {

name: "Beef"

measure: "1kg"

}

1: {

name: "Onions"

measure: "3"

}

2: {

name: "Garlic"

measure: "4 cloves"

}

3: {

name: "Olive oil"

measure: "Dash"

}

4: {

name: "Chorizo"

measure: "300g"

}

5: {

name: "Cumin"

measure: "2 tsp"

}

6: {

name: "Allspice"

measure: "2 tsp"

}

}

}

random_meal (no input)

B. Domain server (s7117-inspections)

All three tools return the same envelope {ok, data, error}; error is null on success.

search_inspections

Search restaurant inspections by facility name, address text, or exact inspection code. query: 2-100 chars; limit: 1-25 (default 10); min_score: 0-100.

☒ Read-only☒ Destructive☒ Idempotent☒ Open-world

query *

golden

limit

5

min_score

90

☐ null

Tool-specific Metadata:

No metadata pairs.

Add Pair

Output Schema:

```
{
  type: "object"
  properties: {
    ok: {
      title: "ok"
    }
  }
}
```

Run Tool

Copy Input

Tool Result: Success

Structured Content:

```
{
  ok: true
  data: {
    count: 5
    results: [
      0: { ... } 6 items
      1: { ... } 6 items
      2: { ... } 6 items
      3: { ... } 6 items
      4: { ... } 6 items
    ]
  }
  error: null
}
```

Valid according to output schema

search_inspections — valid (query = golden, limit = 5, min_score = 90)

get_inspection

Full detail for one inspection (facility, score, inspector, violations).
inspection_code must look like INS-000042.

✓ Read-only ✗ Destructive ✓ Idempotent ✗ Open-world

inspection_code *

INS-000038

Tool-specific Metadata:

No metadata pairs.

Add Pair

Output Schema:

```
{
  type: "object"
  properties: {
    ok: {
      title: "Ok"
    }
  }
}
```

Expand

Run Tool

Copy Input

Tool Result: **Success**

Structured Content:

```
{
  ok: true
  data: {
    id: 38
    inspection_code: "INS-000038"
    facility_name: "FA0649056 - EL NOODLE HOUSE"
    site_address: "1467 ALMADEN EXPY, SAN JOSE, CA 95127"
    score: 87
    created_at: "2026-10-05 04:44:48"
    updated_at: "2026-10-05 04:44:48"
    inspector_id: 7
    inspector_name: "Tara Shah"
    inspector_district: "Cambrian"
    violations: [
      0: { ... } 5 items
      1: { ... } 5 items
      2: { ... } 5 items
      3: { ... } 5 items
    ]
  }
  error: null
}
```

get_inspection — valid (inspection_code = INS-000038)

get_inspection

Full detail for one inspection (facility, score, inspector, violations).
inspection_code must look like INS-000042.

✓ Read-only ✗ Destructive ✓ Idempotent ✗ Open-world

inspection_code *

12345



Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

```
{
  type: "object"
  properties: {
    ok: {
      title: "ok"
    }
  }
}
```



Run Tool

Copy Input

Tool Result: **Success**

Structured Content:

```
{
  ok: false
  data: null
  error: "invalid input for get_inspection: inspection_code: String should match pattern '^INS-\d{6}$'"
}
```



✓ Valid according to output schema

Unstructured Content:

✓ Structured content matches text block

```
{
  ok: false
  data: null
  error: "invalid input for get_inspection: inspection_code: String should match pattern '^INS-\d{6}$'"
}
```



get_inspection — rejected (inspection_code = 12345)

inspection_stats

Aggregate over inspections: count, avg/min/max score, pass rate (score >= 70) and violation count. Optional filters: inspector_id (int >= 1), zip_code (5 digits), facility_query (2-100 chars). Aggregates over 1-4 inspections are blocked by the small-group safety rule.

✓ Read-only ✗ Destructive ✓ Idempotent ✗ Open-world

inspector_id

☐ null

3

zip_code

☒ null

null

facility_query

☒ null

null

Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

```
{
  type: "object"
  properties: {
    ok: {
      title: "ok"
    }
  }
}
```



Run Tool

Copy Input

Tool Result: **Success**

Structured Content:

```
{
  ok: true
  data: {
    filters: {
      inspector_id: 3
    }
    inspection_count: 250
    avg_score: 99.92
    min_score: 88
    max_score: 100
    pass_rate: 1
    violation_count: 8
  }
  error: null
}
```



Inspection_stats — valid (inspector_id = 3)

inspection_stats

Aggregate over inspections: count, avg/min/max score, pass rate (score >= 70) and violation count. Optional filters: inspector_id (int >= 1), zip_code (5 digits), facility_query (2-100 chars). Aggregates over 1-4 inspections are blocked by the small-group safety rule.

✓ Read-only

✗ Destructive

✓ Idempotent

✗ Open-world

inspector_id

3

null

zip_code

95A18

null

facility_query

null

null

Tool-specific Metadata:

No metadata pairs.

Add Pair

Output Schema:

Expand

```
{
  type: "object"
  properties: {
    ok: {
      +i+1a+ "0k"
    }
  }
}
```

Run Tool

Copy Input

Tool Result: Success

Structured Content:

```
{
  ok: false
  data: null
  error: "invalid input for inspection_stats: zip_code: String should match pattern '^\d{5}$'"
}
```

✓ Valid according to output schema

30. inspection_stats — rejected (zip_code = 95A18)

Part 3 - Tool Contracts Under Stress

18. Contracts and rejected calls

The rejected calls are the Part 2B calls. Every input model forbids unknown fields. Validation runs inside `execute_tool` before any database call, so a rejected call is never retried.

search_inspections(screenshots are already added in above section)

Expected input schema:

```
{"type": "object", "additionalProperties": false, "required": ["query"],
```

```
"properties": {
  "query": {"type": "string", "minLength": 2, "maxLength": 100},
  "limit": {"type": "integer", "minimum": 1, "maximum": 25, "default": 10},
  "min_score": {"type": ["integer", "null"], "minimum": 0, "maximum": 100}}
```

Rejected input:

```
{"query": "golden", "limit": 500}
```

Returned error:

```
{"ok": false, "data": null,
  "error": "invalid input for search_inspections: limit: Input should be less than or equal to 25"}
```

Why: limit is capped at 25. A value of 500 would turn a lookup into a bulk export and flood the agent's context.

get_inspection

Expected input schema:

```
{"type": "object", "additionalProperties": false, "required": ["inspection_code"],
  "properties": {"inspection_code": {"type": "string", "pattern": "^INS-\\d{6}$"}}
```

Rejected input:

```
{"inspection_code": "12345"}
```

Returned error:

```
{"ok": false, "data": null,
  "error": "invalid input for get_inspection: inspection_code: String should match pattern '^INS-\\d{6}$'"}
```

Why: "12345" is a bare number, not an inspection code. It is rejected as a format error instead of being sent as a lookup that can never match.

inspection_stats

Expected input schema:

```
{"type": "object", "additionalProperties": false,
  "properties": {
    "inspector_id": {"type": ["integer", "null"], "minimum": 1},
    "zip_code": {"type": ["string", "null"], "pattern": "^\\d{5}$"},
    "facility_query": {"type": ["string", "null"], "minLength": 2, "maxLength": 100}}}
```

Rejected input:

```
{"zip_code": "95A18"}
```

Returned error:

```
{"ok": false, "data": null,
  "error": "invalid input for inspection_stats: zip_code: String should match pattern '^\\d{5}$'"}
```

Why: A ZIP must be 5 digits. "95A18" would silently match nothing and return a misleading empty aggregate, so it is rejected explicitly.

19. Timeouts and bounded exponential-backoff retries

Policy (domain_tools/resilience.py): at most 3 attempts, 2 s timeout per attempt, backoff 0.1 s then 0.2 s ($\times 2$, capped at 1 s). Only transient errors are retried (timeouts, dropped connections, injected faults). After the last attempt the tool returns a clean {ok: false} result.

```

for attempt in range(1, policy.max_attempts + 1):
    result.attempts = attempt
    t0 = time.perf_counter()
    try:
        value = _EXECUTOR.submit(fn).result(timeout=policy.timeout_s)
    except FutureTimeout:
        exc: BaseException = ToolTimeout(f"timed out after {policy.timeout_s:g}s")
    except Exception as e: # noqa: BLE001 - classified below
        exc = e
    else:
        result.attempt_log.append({"attempt": attempt, "outcome": "success",
                                   "ms": round((time.perf_counter() - t0) * 1000, 3)})
        result.ok, result.value = True, value
        break

    transient = _is_transient(exc)
    entry = {"attempt": attempt, "outcome": "transient_error" if transient else "error",
            "error": f"{type(exc).__name__}: {exc}",
            "ms": round((time.perf_counter() - t0) * 1000, 3)}
    result.attempt_log.append(entry)
    result.error = entry["error"]
    if not transient or attempt == policy.max_attempts:
        break
    delay = policy.delay_before(attempt)
    entry["backoff_s"] = delay
    sleep(delay)

result.elapsed_ms = round((time.perf_counter() - start) * 1000, 3)
return result

```

Output of make retry-demo (search_inspections against MySQL):

```

===== [2026-10-05 13:19:27 PDT] make retry-demo (user: rachandeep Kaur) =====
Retry policy: max_attempts=3, timeout=2.0s per attempt, backoff=0.1s x2 (cap 1.0s)
Backend: MySQL s7117_rel

== 1. success on first attempt
attempt 1: success [492.8 ms]
result after 1 attempt(s), 492.8 ms: {"ok": true, "data": "<3 results>", "error": null}

== 2. timeout on attempt 1, success after retry
attempt 1: transient_error (ToolTimeout: timed out after 2s) [2004.2 ms] -> backoff 0.1s
attempt 2: success [3.5 ms]
result after 2 attempt(s), 2110.7 ms: {"ok": true, "data": "<3 results>", "error": null}

== 3. failure after all allowed retries
attempt 1: transient_error (TransientError: injected fault: connection reset by peer) [0.1 ms] -> backoff 0.1s
attempt 2: transient_error (TransientError: injected fault: connection reset by peer) [0.2 ms] -> backoff 0.2s
attempt 3: transient_error (TransientError: injected fault: connection reset by peer) [1.4 ms]
result after 3 attempt(s), 325.8 ms: {"ok": false, "data": null, "error": "search_inspections failed after 3 attempt(s): TransientError: injected fault: connection reset by peer"}

write: /Users/nc/Documents/sisu/Data-260-BW-Git/data260-7117/reports/hw05/raw/retry_demo.json

```

20. Seeded fault injection (VERIFY_SEED = 267117)

Each call is search_inspections against MySQL under the policy above. A FaultInjector seeded with VERIFY_SEED decides on every attempt whether storage fails. Replaying the seed gave an identical success/failure sequence at all three rates. The 150 raw rows are in reports/hw05/raw/fault_injection_calls.csv.


```

51 def run_tool(
52     name: str,
53     inputs: Any,
54     *,
55     repo: Optional[InspectionRepository] = None,
56     policy: RetryPolicy = INTERACTIVE_POLICY,
57 ) -> tuple[Envelope, Optional[RetryResult]]:
58     """execute_tool's logic, also returning the retry trace (attempts, timings)
59     for the Part 3 experiments. Never raises."""
60     spec = TOOLS.get(name)
61     if spec is None:
62         return fail(f"unknown tool '{name}'; available: {'', '.join(TOOLS)}"), None
63     if not isinstance(inputs, dict):
64         return fail(f"invalid input for {name}: inputs must be a JSON object, got {type(inputs).__name__}"), None
65     try:
66         args = spec.input_model.model_validate(inputs)
67     except ValidationError as exc:
68         return fail(format_validation_error(name, exc)), None
69
70     repo = repo if repo is not None else default_repo()
71     trace = call_with_retry(lambda: spec.handler(repo, args), policy)
72     if not trace.ok:
73         if trace.error and trace.error.startswith("LookupError: "):
74             return fail(trace.error.removeprefix("LookupError: "), trace)
75         return fail(f"{name} failed after {trace.attempts} attempt(s): {trace.error}"), trace
76
77     blocked = check_safety_rule(name, trace.value)
78     if blocked:
79         return fail(blocked), trace
80     return ok(trace.value), trace
81
82
83 def execute_tool(
84     name: str,
85     inputs: Any,
86     *,
87     repo: Optional[InspectionRepository] = None,
88     policy: RetryPolicy = INTERACTIVE_POLICY,
89 ) -> str:
90     try:
91         envelope, _trace = run_tool(name, inputs, repo=repo, policy=policy)
92     except Exception as exc: # noqa: BLE001 - last line of defense, must not crash the agent
93         envelope = fail(f"internal error in {name}: {type(exc).__name__}: {exc}")
94     return json.dumps(envelope, default=str)

```

23–24. Offline test runner

code/tests/test_hw05_tools.py uses plain asserts. Each test builds a fresh in-memory fixture repository and passes it to execute_tool through the repo= argument (dependency injection). The agent tests use MockModel. Nothing touches MySQL, Ollama, the network or an API key, so the suite can be re-run any number of times. Run: make test-hw05.

```
===== [2026-10-05 13:31:55 PDT] make test-hw05 (user: rachandeep Kaur) =====
HW5 offline tool tests - Rachandeep Kaur (SID4 7117)
=====
PASS test_search_valid
PASS test_search_rejects_limit_out_of_range
PASS test_search_rejects_unknown_field
PASS test_detail_valid
PASS test_detail_rejects_malformed_code
PASS test_detail_not_found_is_clean_error
PASS test_stats_valid
PASS test_stats_rejects_bad_zip
PASS test_unknown_tool_and_non_object_inputs
PASS test_retry_recovers_after_transient_failure
PASS test_retry_exhausted_returns_clean_error
PASS test_safety_rule_blocks_small_group_aggregate
PASS test_run_agent_stops_at_max_steps_with_mock_model
PASS test_run_agent_normal_completion_with_mock_model
=====
14/14 tests passed
```

Terminal running make test-hw05 — PASS/FAIL per test and 14/14 summary

Part 5 - Agent Loop, Safety Rule, and Reflection

I. Safety rule

Small-group suppression: inspection_stats may not return an aggregate over 1–4 inspections. Statistics over so few rows effectively disclose individual results (a facility's "average" is its own score). A blocked call returns the envelope without raising an exception.

```

MIN_AGGREGATE_GROUP = 5
SAFETY_PREFIX = "SAFETY_RULE_BLOCKED"

_default_repo: Optional[InspectionRepository] = None

def default_repo() -> InspectionRepository:
    global _default_repo
    if _default_repo is None:
        _default_repo = SqlRepository()
    return _default_repo

def check_safety_rule(name: str, data: Any) -> Optional[str]:
    """Return an error message if the result violates the safety rule."""
    if name == "inspection_stats":
        n = data.get("inspection_count", 0)
        if 0 < n < MIN_AGGREGATE_GROUP:
            return (f"{SAFETY_PREFIX}: inspection_stats matched {n} inspection(s); aggregates over "
                    f"fewer than {MIN_AGGREGATE_GROUP} inspections are suppressed because they would "
                    f"disclose individual results. Use get_inspection for a single record or widen the filter.")
    return None

```

```

trace = call_with_retry(lambda: spec.handler(repo, args), policy)
if not trace.ok:
    if trace.error and trace.error.startswith("LookupError: "):
        return fail(trace.error.removeprefix("LookupError: "), trace)
    return fail(f"{name} failed after {trace.attempts} attempt(s): {trace.error}"), trace

blocked = check_safety_rule(name, trace.value)
if blocked:
    return fail(blocked), trace
return ok(trace.value), trace

```

Output of make safety-demo:

```

===== [2025-10-05 13:39:45 PDT] make safety-demo (user: rachandeep Kaur) =====
Safety rule: inspection_stats must cover >= 5 inspections

[ALLOWED] execute_tool('inspection_stats', {"inspector_id": 1})
-> {"ok": true, "data": {"filters": {"inspector_id": 1, "inspection_count": 250, "avg_score": 99.81, "min_score": 84, "max_score": 100, "pass_rate": 1.0, "violation_count": 19}, "error": null}}

[BLOCKED] execute_tool('inspection_stats', {"facility_query": "FA0183034"})
-> {"ok": false, "data": null, "error": "SAFETY_RULE_BLOCKED: inspection_stats matched 1 inspection(s); aggregates over fewer than 5 inspections are suppressed because they would disclose individual results. Use get_inspection for a single record or widen the filter."}

```

II. Agent loop

run_agent(user_input) in domain_tools/agent.py uses qwen3:8b through Ollama. On each turn the model replies with one JSON object: a tool call or a final answer. Every tool call goes through execute_tool. The loop stops on final_answer, safety_block or max_steps, and appends every step to reports/hw05/raw/agent_runs.jsonl: the tool, its inputs, the result, timings and the stop reason.

```

log({"event": "start", "user_input": user_input, "max_steps": max_steps})
step = tool_calls = 0
stop_reason, final_answer = "max_steps", None

while step < max_steps:
    step += 1
    t0 = time.perf_counter()
    try:
        raw = model.complete(messages)
    except Exception as exc: # noqa: BLE001 - Ollama down / model missing
        stop_reason, final_answer = "model_error", f"{type(exc).__name__}: {exc}"
        log({"event": "step", "step": step, "error": final_answer})
        break
    model_ms = round((time.perf_counter() - t0) * 1000, 1)
    messages.append({"role": "assistant", "content": raw})

    try:
        reply = json.loads(raw)
        action = reply.get("action")
    except (json.JSONDecodeError, AttributeError):
        reply, action = None, None

    if action == "final":
        final_answer = str(reply.get("answer", ""))
        stop_reason = "final_answer"
        log({"event": "step", "step": step, "model_ms": model_ms, "action": "final", "answer": final_answer})
        break

    if action != "tool":
        feedback = ('Invalid reply. Respond with one JSON object: {"action": "tool", ...} '
                    'or {"action": "final", "answer": ...}.')
        log({"event": "step", "step": step, "model_ms": model_ms, "action": "invalid", "raw": raw[:500]})
        messages.append({"role": "user", "content": feedback})
        continue

    tool, inputs = reply.get("tool"), reply.get("inputs", {})
    tool_calls += 1
    t1 = time.perf_counter()
    result_json = execute_tool(str(tool), inputs, repo=repo)
    result = json.loads(result_json)
    log({"event": "step", "step": step, "model_ms": model_ms, "action": "tool", "tool": tool,
        "inputs": inputs, "result": result, "tool_ms": round((time.perf_counter() - t1) * 1000, 1)})

    if not result["ok"] and str(result["error"]).startswith(SAFETY_PREFIX):
        stop_reason = "safety_block"
        final_answer = f"Request blocked by safety rule: {result['error']}"
        break
    messages.append({"role": "user", "content": f"TOOL_RESULT {tool}: {result_json[:MAX_RESULT_CHARS]}"})

summary = {"run_id": run_id, "scenario": scenario, "model": model.name, "user_input": user_input,
    "stop_reason": stop_reason, "steps": step, "tool_calls": tool_calls,
    "max_steps": max_steps, "final_answer": final_answer}
log({"event": "stop", **{k: v for k, v in summary.items() if k not in ("run_id", "scenario", "model")}})
summary["events"] = events
return summary

```

III. Additional tests

Two tests were added to the Part 4 suite. Both pass offline:

Test	Checks
------	--------

test_safety_rule_blocks_small_group_aggregate	execute_tool returns {ok: false, data: null, error: SAFETY_RULE_BLOCKED...} for a 1-inspection aggregate; a 7-inspection aggregate is allowed
test_run_agent_stops_at_max_steps_with_mock_model	A MockModel that only ever calls tools stops at max_steps = 3 with steps = 3, tool_calls = 3 and a final 'stop' log line

IV. Metrics (qwen3:8b, local Ollama)

Scenario	max_steps	Steps	Tool calls	Stop reason
search — GOLDEN facilities scoring ≥ 90	6	2	1	final_answer
detail — violations of INS-000042	6	2	1	final_answer
aggregate — inspector 3 average and pass rate	6	2	1	final_answer
safety — stats for one facility	6	1	1	safety_block
max_steps — compare inspectors 1–5	3	3	3	max_steps

I checked every final answer against the tool result in agent_runs.jsonl. For example, the detail answer (Bay BBQ, violation 29, score 98, inspector Elena Rivera) matches get_inspection exactly.

V. Reflection

Run: run_id = eb9f2c679b81 · scenario max_steps · model qwen3:8b · stop reason max_steps · 3 steps · 3 tool calls

The user instructed the agent to find out which inspector 1 through 5 had the highest average score on their inspections, with each inspection_stats request per inspector. For this scenario the harness was launched with max_steps = 3 and reported the launch event along with the input and the above limit.

Step 1. The harness provided the system prompt (three tool schemas and the JSON reply format) and the question to the model qwen3:8b. After 16.5 seconds the model replied with {"action": "tool", "tool": "inspection_stats", "inputs": {"inspector_id": 1}}. The harness passed it to the execute_tool function which verified the input and executed the MySQL query according to the retry policy and the safety rule. The aggregation concerned 250 inspections and was safe according to the rule; the call was answered ok: true with the average equal to 99.81 and took 17 ms.

Steps 2 and 3. The same process occurred in case of inspector 2 (99.91) and inspector 3 (99.92) where each step of the model was more time-consuming (22.3 s and then 24.1 s) since the dialogue became longer. No call was blocked or unsuccessful.

Why it ended. After step 3 the number of steps was equal to `max_steps`, which stopped the while loop. The model could not take another turn and make the last call and, thus, could not call inspectors 4 and 5. The harness provided `stop_reason = "max_steps"`, `steps = 3` and `tool_calls = 3`, as well as empty `final_answer`. It could not estimate which inspector was the best among the first three.

It is the effect of the ceiling that operates: the task which needs 5 tool calls cannot be completed in 3 turns and, hence, the cost of the run is limited by 3 model calls.

AI Use

1. What I used an AI assistant for, and what I did myself

AI assistant:

- Drafted the HW5 migration (inspectors table, FK with ON DELETE RESTRICT), the inspectors router and the extended records router,
- Wrote the two MCP servers, the shared tool layer (envelope, retry/timeout policy, seeded fault injection, `execute_tool`, safety rule, agent loop), the offline test runner, the experiment scripts, `verify.py` and the Makefile targets.
- Drafted the report skeleton, METRICS tables and this file.

What I did myself:

- Ran the migration, the API, the React client and every make target, and analysed all Postman apis, database, MCP Inspector, and Redux functionality and the Redux Toolkit slices/thunks for the React client.
- Chose the related entity (inspectors) and reviewed the safety rule (small-group suppression).
- Read the generated code and checked the outputs against the database.

2. One AI-produced output that was wrong, and one thing I verified independently

Wrong: the Makefile pinned the MCP SDK for mcp dev as `--with "mcp[cli]>=1.9,<2"`. That works with the new Inspector 2.x, but the classic Inspector (0.22, which shows inputs and output together for screenshots) could never connect: it stayed "Disconnected" with no error in the UI.

Verified independently: For each scenario, I compared the answer with the tool result logged in `agent_runs.jsonl` such as the detail run's "Bay BBQ, violation code 29, score 98, inspector Elena Rivera (Alum Rock)" matches the `get_inspection` result for INS-000042.

3. How I detected the problem / verified the result

The Inspector proxy log showed the server being started with `args=run,--with,mcp[cli],[object Object],=1.9,,[object Object],2,....`. The classic Inspector splits the argument string like a shell, so `>` and `<` were parsed as redirection operators and the version pin was destroyed before uv ever ran.

4. What I changed and why it works now

The pin is now written `mcp[cli]~=1.9`, which means the same thing (≥ 1.9 and < 2) without `<` or `>`. Both `make mcp` (`mcp dev`) and `make inspector` (classic Inspector) now connect and run tools; I confirmed it by running `meals_by_ingredient` (chicken) and the rejected `search_inspections` (limit 500) call in the Inspector.